

Innovaciones educativas en el ambito edificatorio

Coordinadores:

Rafael Vicente Lozano Díez

Sofía Herrero del Cura

Amparo Verdú vázquez

Dykinson, S.L.

INNOVACIONES EDUCATIVAS EN EL AMBITO
EDIFICATORIO

INNOVACIONES EDUCATIVAS EN EL AMBITO EDIFICATORIO

Coordinadores

Rafael Vicente Lozano Díez

Sofía Herrero del Cura

Amparo Verdú Vázquez

Dykinson, S.L.

2022

Cómo citar esta obra:

Lozano-Díez, R.V.; Herrero-Del-Cura, S; Verdú-Vázquez, A. (Coordinadores).

Innovaciones Educativas en el ámbito edificatorio.

Dykinson.

INNOVACIONES EDUCATIVAS EN EL AMBITO EDIFICATORIO

Fotografía de cubierta: J.M. Arroyo

© de los textos: los autores

© de la presente edición: Dykinson S.L. Madrid -
2022

ISBN: 978-84-1122-751-3

NOTA EDITORIAL: Las opiniones y contenidos publicados en esta obra son de responsabilidad exclusiva de sus autores y no reflejan necesariamente la opinión de Dykinson S.L ni de los editores o coordinadores de la publicación; asimismo, los autores se responsabilizarán de obtener el permiso correspondiente para incluir material publicado en otro lugar.

ÍNDICE

INTRODUCCIÓN	7
CAPÍTULO 1. INMERSIÓN EN EL MUNDO PROFESIONAL DE LA EDIFICACIÓN: INNOVACIÓN EDUCATIVA MEDIANTE VISITAS TÉCNICAS A ENTORNOS LABORALES	9
D. Villanueva Valentín-Gamazo	
G. Arcones Pascual	
S. Bellido Blanco	
CAPÍTULO 2. ANÁLISIS DE EXPERIENCIAS DE LIDERAZGO Y TRABAJO COLABORATIVO EN LA ENSEÑANZA DE TOPOGRAFÍA	35
Cristina Torrecillas Lozano	
Eduardo Vázquez-López	
Laura García-Ruesgas	
CAPÍTULO 3. ASOCIACIÓN ENTRE VISTAS ORTOGRÁFICAS Y OBJETOS 3D EN CeDG: PRUEBA DE CONCEPTO	57
Manuel Prado Velasco	
Laura García-Ruesgas	
CAPÍTULO 4. INNOVATIVE PROCESS OF CREATING SMALL ELEMENTS FOR BUILDING	71
José Antonio Hernández Torres	
Ángel Mariano Rodríguez-Pérez	
Julio José Caparrós Mancera	
CAPÍTULO 5. FACILITANDO LA CARACTERIZACIÓN Y NUEVOS MATERIALES DE CONSTRUCCIÓN A LOS ALUMNOS DE BACHILLERATO	83
Álvaro Alonso Díez	
Raquel Arroyo Sanz	
Lourdes Alameda Cuenca-Romero	
Verónica Calderón Carpintero	
CAPÍTULO 6. INSTAGRAM COMO HERRAMIENTA EDUCATIVA.....	93
Guadalupe Dorado Escribano	
Jorge Pablo Díaz Velilla	
CAPÍTULO 7. SERIOUS GAME APLICADO A LOS ESTUDIOS UNIVERSITARIOS DE DIRECCIÓN DE PROYECTOS	107
Manuel Botejara-Antúnez	
Pablo Garrido-Píriz	
Jaime González-Domínguez	
Gonzalo Sánchez-Barroso	
Justo García-Sanz-Calcedo	

CAPÍTULO 3. ASOCIACIÓN ENTRE VISTAS ORTOGRÁFICAS Y OBJETOS 3D EN CeDG: PRUEBA DE CONCEPTO

Manuel Prado Velasco

*Departamento de Ingeniería Gráfica, Escuela Técnica Superior de
Ingeniería, Universidad de Sevilla*

Laura García-Ruesgas

*Departamento de Ingeniería Gráfica, Escuela Técnica Superior de
Ingeniería, Universidad de Sevilla*

1. INTRODUCCIÓN

La representación gráfica de elementos tridimensionales requiere el empleo de un lenguaje técnico que proporcione precisión y facilite su análisis y comunicación con procedimientos aplicados sobre formas planas. La geometría descriptiva, permite la representación y análisis de un sistema 3D a partir de sus proyecciones en el plano (Graves, 1912). Esta disciplina, apoyada en las geometrías proyectiva y métrica (Millman et al., 1991), definió los fundamentos del lenguaje gráfico del dibujo técnico moderno.

Los procedimientos de representación gráfica de sistemas tridimensionales disponían de un corpus importante de estándares en la década de 1970, que han ido evolucionando, confirmando el cambio de paradigma en las técnicas gráficas de representación en la ingeniería, debido a la emergencia de los sistemas de diseño asistido por computador (CAD) (Weisberg, 2008).

En la actualidad, la tecnología CAD posibilita la representación de sistemas tridimensionales, permitiendo construir modelos paramétricos computacionales (Migliari, 2012), facilitando la modificación de sus propiedades geométricas y la propagación automática al resto del modelo, el control del proceso de diseño y fabricación (Fitter et al., 2014), (Machado et al., 2019) y el estudio del comportamiento del sistema mediante la conexión con su modelo dinámico (Mejía-Gutiérrez et al., 2017). Sin embargo, existen limitaciones del CAD en relación a la geometría descriptiva (Stachel et al., 2007), (Migliari, 2012).

La técnica de modelado computacional fundamentada en la geometría descriptiva o Computer extended Descriptive Geometry (CeDG) surge como complemento a la metodología Computer Assisted Design (CAD) actual, integrando a los modelos de geometría 3D computacionales la competencia de resolución de problemas de análisis espacial, inherente a la geometría descriptiva (Prado-Velasco et al., 2021). CeDG aúna la capacidad resolutoria que

brindan los procedimientos de la geometría descriptiva, con la destreza del software de geometría dinámica (DGS) para implementar modelos gráfico-matemáticos con parametrización dinámica.

Las primeras herramientas DGS aparecen a principios de la década de 2000. Estas se definen como un software de geometría computacional orientado al análisis de problemas geométricos combinando álgebra y geometría (Preiner, 2008). Una característica esencial de las herramientas DGS es la conexión de las descripciones algebraicas con las geométricas. Un sistema DGS muy extendido es GeoGebra, que incluye módulos para el sistema algebraico computacional (CAS), hoja de cálculo y estadística (Hohenwarter, 2009). GeoGebra es el DGS utilizado como herramienta para implementar CeDG.

Diversos estudios han demostrado el potencial de CeDG sobre GeoGebra en el desarrollo de piezas de calderería que salvan algunas limitaciones de los sistemas de CAD actuales, así como en el modelado de sistemas mecánicos, donde los parámetros geométricos están relacionados con parámetros funcionales mediante relaciones algebraicas que constituyen parte del modelo CeDG y permiten obtener los valores de diseño durante el proceso de modelado (Prado Velasco et al., 2021), (Prado Velasco et al., 2021).

GeoGebra soporta varios lenguajes de programación para la adición de nuevas funciones al sistema, entre ellos Javascript. Adicionalmente, y considerando que la mayor parte del código GeoGebra sigue licencia de código abierto, es posible realizar extensiones de su funcionalidad sobre dicho código. El empleo de Java como lenguaje para el desarrollo de GeoGebra, facilita por otro lado el despliegue de GeoGebra en múltiples plataformas, gracias a la tecnología de máquina virtual Java. Actualmente existen versiones para Windows, Linux, MacOS, Android, iPad, e incluso una versión basada en web en la versión 5 de HTML.

Una de las limitaciones que presenta la actual versión de CeDG sobre GeoGebra, denominada **CeDG Base**, es la falta de asociación entre las vistas ortográficas de un objeto con el propio objeto 3D. Esta asociación solo existe en la mente del creador del modelo, de forma que todos los procedimientos diédricos deben aplicarse manualmente por el usuario. Resulta de gran interés que la versión base evolucione a una nueva versión **CeDG v1** en la que la técnica CeDG asocie las vistas con los objetos espaciales que las generan. Esta nueva versión facilitaría la ejecución de los procedimientos de geometría descriptiva.

En este trabajo se presenta una prueba de concepto que evalúa la posibilidad de incorporar a GeoGebra la relación entre vistas ortográficas y objetos 3D asociados mediante Javascript.

2. DISEÑO CONCEPTUAL DE CeDG

Un modelo CeDG se define por medio de una secuencia ordenada de entidades gráfico-algebraicas que mantienen relaciones de dependencia con

entidades anteriores en dicha secuencia. CeDG sigue un diseño conceptual organizado en tres capas, ver Figura 1.

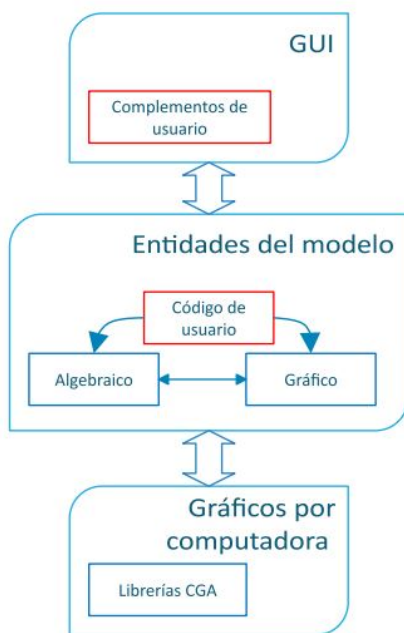
La capa *interface gráfica de usuario* (GUI) incluye un conjunto de funcionalidades básicas para la creación de objetos gráfico-algebraicos que pueden clasificarse en:

- Ventanas gráfica y algebraica para la visualización y edición de las entidades que definen el modelo. El usuario puede acceder a cualquier entidad existente del modelo mediante su formalismo algebraico o su formalismo gráfico (si este segundo existe), de forma indistinta.

- Capacidad para definir y gestionar los objetos gráfico-algebraicos del modelo, incluyendo puntos, curvas y otros elementos. Esta funcionalidad incluye la adición de relaciones de dependencia con objetos previos. La entrada y edición de estos objetos y relaciones incluye también el modo gráfico para un conjunto de objetos.

- Configuración del área gráfica, referencias e interacción con el usuario. Estas incluyen la rejilla, indicadores de inferencia (coincidencia, relaciones geométricas, etc.), zoom, ajuste del área, y obtención de medidas, entre otras.

Figura 1: Diseño conceptual de la técnica CeDG



Fuente. Elaboración propia

La capa GUI se puede ampliar mediante complementos añadidos por el usuario, como indica la figura 1. De esta manera, es posible añadir herramientas gráficas a comandos que no dispongan de estas.

La capa de *entidades del modelo* permite la creación y modificación de las entidades gráfico-algebraicas en una secuencia ordenada, incluyendo las dependencias entre cada una y un número cualquiera de entidades anteriores en la secuencia. Las entidades admiten la adición de código de usuario que genere otras entidades posteriores en la secuencia, al ser activadas de alguna otra forma. De igual manera es posible añadir nuevos comandos al software que implementa CeDG, permitiendo su evolución. Para ello, existen diferentes técnicas, pero en general es posible el acceso a las funciones internas del software mediante su Application Programming Interface (API), al que se puede acceder mediante código Scripting, generado mediante GGBScript (código basado en comandos de GeoGebra) y mediante Javascript. El código se puede asociar a una entidad gráfico-algebraica o ejecutarse al abrir el modelo (fichero tipo ggb).

El modelado CeDG de un sólido 3D utiliza los procedimientos del sistema de representación basado en geometría descriptiva que se elija. En este trabajo se utiliza DiédricoDirecto por su amplia difusión en la ingeniería.

La construcción de un modelo CeDG de un sistema 3D es similar a la representación de dicho sistema en diédrico sobre papel. Esto es, el experto genera las proyecciones ortogonales de los elementos geométricos que definen las superficies, líneas de contorno y otros elementos geométricos de la pieza y utiliza los procedimientos gráficos del diédrico para manipular la pieza y resolver los problemas geométrico - funcionales presentes en el estudio.

Como los procedimientos gráficos del diédrico están asociados a resultados algebraicos, los modelos CeDG proporcionan una estructura formal natural para su implementación computacional que añade importantes ventajas con relación a la representación en papel. Además de la precisión y simplificación de los procedimientos constructivos, los modelos CeDG añaden la capacidad de parametrización y otras ventajas conceptuales asociadas al empleo de geometría dinámica.

Los parámetros de un modelo CeDG pueden ser tratados mediante literales fijos o mediante variables. Los parámetros asociados a variables pueden modificarse de manera dinámica mediante instrumentos GUI interactivos. El cambio dinámico de un parámetro afecta a las entidades gráfico-algebraicas que dependan del parámetro, directa o indirectamente (a través de otras entidades). Por ello, un modelo CeDG evoluciona con los parámetros de manera síncrona, es decir permite parametrización dinámica.

La capa de *gráficos por computadora* está encargada de la generación de las imágenes gráficas asociadas a las entidades gráfico-algebraicas. Esta utiliza a su vez librerías que implementan algoritmos de geometría computacional (CGA), que son dependientes de las funciones requeridas por el software. Como CeDG se implementa sobre un software de geometría dinámica (GeoGebra en este

trabajo), las librerías CGA que utiliza presentan diferencias con respecto a las librerías CGA de las herramientas de CAD.

3. METODOLOGÍA

En primer lugar, se va a considerar que cualquier sistema 3D se puede descomponer en un conjunto de subsistemas 3D, y cada uno de estos subsistemas a su vez, se puede descomponer en otro conjunto de subsistemas, hasta alcanzar el nivel deseado (enfoque top – down con un número arbitrario de niveles).

En segundo lugar, se considera que la proyección de un sistema 3D compuesto de n subsistemas 3D se puede componer de n proyecciones (sobre un cierto plano y válido tanto para una proyección cilíndrica como cónica), cada una asociada a uno de los n subsistemas. Esta consideración no implica que la presentación o rendering de la proyección compuesta de las n proyecciones sea la suma simple de las n proyecciones. El tratamiento de este último asunto queda fuera del objeto de este trabajo.

Bajo las consideraciones anteriores, para evaluar la capacidad de CeDG sobre GeoGebra en la incorporación de la asociación entre un sistema 3D y sus proyecciones, es suficiente con utilizar un sistema básico o primitiva 3D.

Con este fin, se ha definido un modelo CeDG de una recta, caracterizada por diferentes vistas, sobre el que se implementará la lógica de asociación vistas objetos 3D en Javascript. Este modelo define la prueba de concepto mediante la que se evalúa la posibilidad de incorporar a GeoGebra la relación entre vistas ortográficas y objetos 3D asociados.

Los resultados se organizan en dos secciones. La primera describe el diseño y código implementado, y la segunda los resultados de ejecución sobre un layout definido por la ventana gráfica acoplada a la ventana algebraica.

4. RESULTADOS

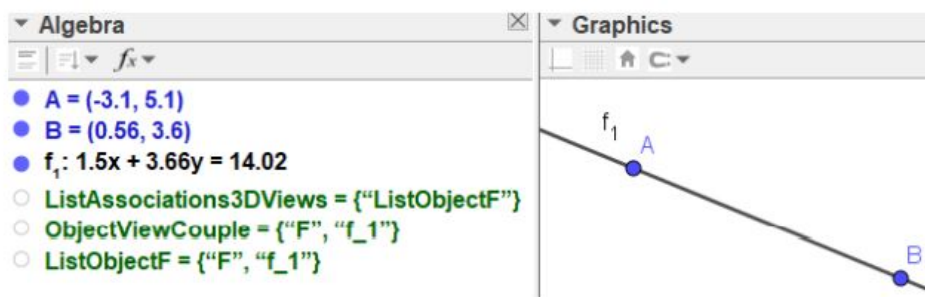
4.1. Diseño y codificación

El código desarrollado hace uso del objeto computacional List de GeoGebra, que puede contener cualquier secuencia de entidades gráfico-algebraicas. Podemos por tanto definir una variable lista para cada objeto espacial existente. Esta variable se define dinámicamente cuando se crea la primera vista del objeto. Así la lista $ListObjectF = \{F, f_1\}$ sería construida cuando se crea la entidad f_1 (primera vista ortográfica) del objeto espacial F . La segunda variable lista $ListAssociations3DViews$ se define como una lista global que incluye todas las variables listas de objetos espaciales.

La figura 2 aclara estos conceptos, mostrando la situación inicial de un modelo CeDG donde solo se ha creado la primera vista ortográfica f_1 de un primer objeto espacial F (recta en 3D) y contiene las dos listas definidas

ListObjectF y ListAssociations3DViews. El código Javascript desarrollado para implementar una prueba de concepto basada en estas variables sobre un modelo CeDG se presenta a continuación.

Figura 2: Estado inicial de un modelo CeDG definido por la vista f1 asociada con el objeto espacial F



Fuente. Elaboración propia

Cuando se quiere añadir otra vista ortográfica del objeto espacial F, el software CeDG comprueba si la lista del objeto espacial F ya existe (ListObjectF). Si la respuesta es afirmativa añade la nueva vista ortográfica a esa lista. En caso contrario, creará la lista del objeto espacial y la inicializará con la vista ortográfica introducida, además de añadirla a la lista global ListAssociations3DViews que mantiene las listas de objetos espaciales existentes en el modelo.

Este proceso se codifica por medio de la función ggbOnInit() de GeoGebra, que se ejecuta simplemente al cargar el fichero y lo que hace básicamente es:

- Comprueba si existe la entidad ListAssociations3DViews. Si no existe la crea.
- Comprueba si existe la entidad ObjectViewCouple. Si no existe la crea.
- Asocia la función onAdd al evento de Add.

La función ggbOnInit() es llamada por el software nada más cargar el fichero del modelo CeDG (fichero .ggb) y permite inicializar el software, creando ListAssociations3DViews y la lista adicional ObjectViewCouple. ObjectViewCouple es una lista auxiliar que se carga con el par (objeto 3D, vista ortográfica asociada) durante el proceso de creación de una nueva entidad vista algebraica. La figura 2 muestra a ObjectViewCouple cargada con $\{F, f_1\}$, que corresponde a la última vista ortográfica definida. La función Javascript ggbOnInit() se muestra en el código 2.1 (ver Figura 3).

ggbOnInit() utiliza el método evalCommand() de ggbApplet para enviar comandos al núcleo de GeoGebra. Esta es la manera en la que se utiliza la Application Programming Interface (API) de GeoGebra desde Javascript. La última línea define onAdd() como la función de usuario que será llamada por el software cada vez que se crea una nueva entidad gráfico-algebraica. La creación de la entidad origina un evento en el software que provoca la ejecución de la función asociada al tipo de evento. Esa función se define mediante el método registerAddListener() de ggbApplet pertenecientes a GeoGebra.

Figura 3: Código fuente 2.1: ggbOnInit() function

```

1 function ggbOnInit() {
2   if (!ggbApplet.exists("ListAssociations3DViews")) {
3     ggbApplet.evalCommand("ListAssociations3DViews = {}")
4   };
5   if (!ggbApplet.exists("ObjectViewCouple")) {
6     ggbApplet.evalCommand("ObjectViewCouple = {}")
7   };
8   ggbApplet.registerAddListener("onAdd");
9 }

```

Fuente. Elaboración propia

La función onAdd() del código 2.2 (ver Figura 4) se llama cada vez que se crea una nueva entidad gráfico-algebraica en el modelo. Lo que hace esta función básicamente es:

- Desactiva onAdd, para no ser interrumpida hasta finalizar el proceso.
- Lee la segunda entidad (vista del objeto) de la lista ObjectViewCouple y lo compara con el argumento.
- Si son iguales, esto significaría que se está introduciendo una vista nueva de un objeto. En este caso llama a searchAssociatedList con el nombre del objeto y de la vista a asociar. En caso contrario no hace nada.
- Vuelve a registrar onAdd al terminar. La lectura de la lista ObjectViewCouple la realiza llamando a la API de GeoGebra, mediante el objeto ggbApplet.

El argumento de entrada, name, es el nombre de la nueva entidad creada. onAdd() compara name con el segundo elemento de ObjectViewCouple. Si ambos coinciden la función procederá a insertar la nueva vista ortográfica en la lista de vistas del objeto asociado.

Antes de crear la entidad correspondiente a la nueva vista ortográfica, la lista auxiliar ObjectViewCouple se carga con (Nombre objeto espacial, Nombre

vista nueva). Por ejemplo, para añadir la nueva vista f2 del objeto espacial F, ObjectViewCouple se carga con {F, f2}. Esta definición se realiza por medio de una herramienta GUI que no se presenta en este trabajo.

Figura 4: Código fuente 2.2: OnAdd() function

```
1 function onAdd(name) {
2   // Avoid new calls during the process
3   ggbApplet.unregisterAddListener("onAdd");
4   cmd = "CEDGstr = ObjectViewCouple(2)";
5   ggbApplet.evalCommand(cmd);
6   str2 = ggbApplet.getValueString("CEDGstr") + "";
7   ggbApplet.evalCommand("Delete(CEDGstr)");
8   if (str2 == name) {
9     cmd = "CEDGstr = ObjectViewCouple(1)";
10    ggbApplet.evalCommand(cmd);
11    str1 = ggbApplet.getValueString("CEDGstr") + "";
12    ggbApplet.evalCommand("Delete(CEDGstr)");
13    searchAssociatedList(str1, str2);
14  };
15  ggbApplet.registerAddListener("onAdd");
16 };
```

Fuente. Elaboración propia

La citada inserción se realiza por la función searchAssociatedList, ejecutada en la línea 13 del Código 2.2. El código searchAssociatedList (código 2.3, Figura 5) recibe el nombre del objeto espacial (spacename) y el nombre de la vista ortográfica asociada (viewname), busca la lista asociada al objeto espacial recibido en la lista global ListAssociations3DViews (ListObjectF en el ejemplo de la figura 2), e inserta la nueva vista ortográfica. Una vez encontrada la lista buscada, añade la nueva vista. Si no encuentra la lista del objeto espacial, esto significa que se está insertando la primera vista ortográfica de dicho objeto y por tanto creará la lista del objeto espacial y apuntará su nombre en la lista global ListAssociations3DViews. Se emplea también el objeto ggbApplet.

Se ha simplificado el código de la función searchAssociatedList tratando solo el caso en que la lista del objeto espacial existe (condición bool = TRUE, línea 15).

Figura 5: Código fuente 2.3: searchAssociatedList function

```

1 function searchAssociatedList(spacename, viewname) {
2   // Search and manage list of associated views
3   cmd = "Length(ListAssociations3DViews)";
4   nLists = parseInt(ggbApplet.getValue(cmd));
5   for (var i = 1; i <= nLists; i++) {
6     cmd = "CEDGstr = ListAssociations3DViews(" + i.toString() + ")";
7     ggbApplet.evalCommand(cmd);
8     viewList = ggbApplet.getValueString("CEDGstr") + "";
9     ggbApplet.evalCommand("Delete(CEDGstr)");
10    cmd = "CEDGbool = (\\" + spacename + "\" == \" + viewList + "(1))";
11    ggbApplet.evalCommand(cmd);
12    bool = ggbApplet.getValue("CEDGbool");
13    ggbApplet.evalCommand("Delete(CEDGbool)");
14    // If the list is found
15    if (bool) {
16      // Prepare new list views
17      views.push(spacename);
18      viewListStr = "\\" + views[0] + "\\";
19      cmd = "Length(\" + viewList + \")";
20      nViews = parseInt(ggbApplet.getValue(cmd));
21      for (var j = 2; j <= nViews; j++) {
22        cmd = "CEDGstr = \" + viewList + "(" + j.toString() + ")";
23        ggbApplet.evalCommand(cmd);
24        str = ggbApplet.getValueString("CEDGstr") + "";
25        ggbApplet.evalCommand("Delete(CEDGstr)");
26        views.push(str);
27        viewListStr = viewListStr + ",\\" + views[j - 1] + "\\";
28      };
29      // Adds the new view object
30      views.push(viewname);
31      viewListStr = viewListStr + ",\\" + views[nViews] + "\\";
32      cmd = "Delete(\" + viewList + \")";
33      ggbApplet.evalCommand(cmd);
34      cmd = viewList + " = {" + viewListStr + "}";
35      ggbApplet.evalCommand(cmd);
36      break;
37    };
38  };
39 };

```

Fuente. Elaboración propia

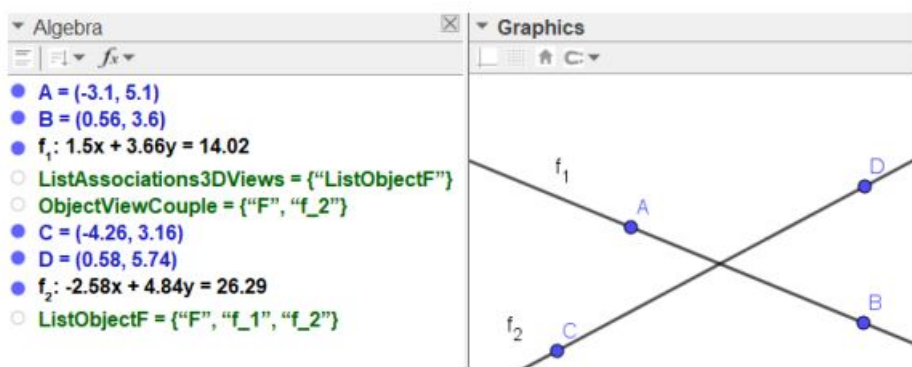
4.2. Evaluación del modelo

El modelo comienza con una primera proyección de la recta (objeto F). Las variables se inicializan al entrar en el modelo (abrir el fichero) mostrando el código que aparece en la figura 2. Como se indica en el apartado anterior, la lista auxiliar se debe cargar con el nombre de la nueva vista que se va a crear y asociar al objeto F o recta.

La figura 6 muestra la situación del modelo CeDG de la figura 2 al introducir la segunda vista ortográfica f2 del objeto espacial F (recta en 3D). Se verifica de esta forma la posibilidad de efectuar la entrada de nuevas vistas ortográficas,

confirmando como el sistema es capaz de gestionar adecuadamente sus eventos, adición de nuevos objetos y detección de aquel objeto que corresponde a una nueva vista de F para asociarla al objeto. Esta prueba de concepto muestra por tanto la posibilidad de asociar vistas ortográficas a objetos espaciales en GeoGebra mediante código Javascript.

Figura 6: Estado final del modelo CeDG tras añadir la segunda vista f2 asociada al objeto espacial F



Fuente. Elaboración propia

4.3. Discusión de los resultados

La prueba de concepto demuestra la posibilidad de resolver la detección de entidades gráfico – algebraicas y asociarlas a los objetos 3D, gestionando los mensajes que genera la interface de GeoGebra. La prueba demuestra la capacidad de GeoGebra para gestionar esta primera etapa, necesaria para evolucionar a CeDG v1.

La prueba de concepto presenta limitaciones. La más importante es la necesidad de cargar el código Javascript en el fichero GGB sobre el que se creará el modelo CeDG.

Pese a ello, al haber demostrado la posibilidad de acceder a las características del núcleo de GeoGebra que son necesarias para la evolución indicada mediante Javascript, tenemos garantía de poder disponer de ellas utilizando Java, que es el código fuente de desarrollo de GeoGebra.

Su implementación en Java define uno de los siguientes pasos en la línea de investigación y desarrollo en CeDG.

5. CONCLUSIONES

Esta investigación presenta una técnica de modelado computacional basada en la geometría descriptiva que surge como complemento a la metodología CAD para integrar los modelos de geometría 3D computacionales con la capacidad de resolución de problemas de análisis espacial, inherente a la Geometría Descriptiva.

Puede afirmarse que el objetivo principal de este trabajo, la posibilidad de incorporar la asociación entre vistas ortográficas y objetos 3D en GeoGebra, queda demostrado con la prueba de concepto que se presenta. En ella se verifica la capacidad de Javascript para gestionar la lógica de esa asociación.

Las limitaciones de la implementación observadas, como la necesidad de cargar el código en el fichero GGB que define el modelo CeDG, pueden superarse al emplear Java como lenguaje de programación. Denominamos **CeDG v1** a una futura versión de GeoGebra que implemente de forma automática la asociación estudiada¹.

Algunas de las futuras líneas de trabajo que se contemplan consisten en la automatización de los procedimientos básicos de geometría descriptiva que permiten el giro con relación a un eje, el cambio de la dirección de proyección (vistas auxiliares) y el abatimiento de planos **CeDG v2**. Este nivel permite visualizar un objeto en diferentes posiciones para inspeccionar o para facilitar la aplicación de métodos geométricos. También facilita la obtención de perspectivas axonométricas, tanto ortogonales como oblicuas.

La futura versión **CeDG v3 y posteriores** incluyen el concepto de sólido y procedimientos geométricos más avanzados de acuerdo a especialización, tales como homologías, sombras, intersecciones de superficies y otros.

REFERENCIAS BIBLIOGRÁFICAS

Fitter, H.N., Pandey, A.B., Patel, D.D. and Mistry, J.M. (2014). A Review on Approaches for Handling Bezier Curves in CAD for Manufacturing, *Procedia Engineering*, 97, 1155-1166.

<https://doi.org/10.1016/j.proeng.2014.12.394>

Graves, O.M.C. (1912). *Orthographic projection: the elementary principles of orthographic projection, with their applications to technical drawing*, Eschenbach brothers. University of Chicago.

¹ Las extensiones previstas de GeoGebra respetarán su licencia, actualmente libre para uso no comercial, sujeto a los términos del "Geogebra Non-Commercial License Agreement", www.geogebra.org/license.

Hohenwarter, J. and Hohenwarter, M. (2009). Geogebra Classic Manual. [http://docplayer.es/576032-Documento-de-ayuda-de-geogebra-manual-oficial-de-la-version-3-2-markus-hohenwarter-y-judith-h\(2009\)ohenwarter-www-geogebra-org.html](http://docplayer.es/576032-Documento-de-ayuda-de-geogebra-manual-oficial-de-la-version-3-2-markus-hohenwarter-y-judith-h(2009)ohenwarter-www-geogebra-org.html)

Machado, F., Malpica, N. and Borromeo, S. (2019). Parametric CAD modeling for open source scientific hardware: Comparing OpenSCAD and FreeCAD Python scripts, *PLoS One*, 14 (12). <https://doi.org/10.1371/journal.pone.0225795>

Mejía-Gutiérrez, R. y Carvajal-Arango, R. (2017). Design Verification through virtual prototyping techniques based on Systems Engineering, *Research in Engineering Design*, 28(4), 477-94. DOI:10.1007/s00163-016-0247-y

Migliari, R. (2012). Descriptive Geometry: From its Past to its Future, *Nexus Network Journal*, 14(3), 555-571. <https://doi.org/10.1007/s00004-012-0127-3>

Millman, R.S. y Parker, G.D. (1991). *A Metric Approach with Models*, Springer-Verlag. New York.

Prado-Velasco, M., Ortiz Marín, R., García Ruesgas, L. and del Río Cidoncha, M.G. (2021). Graphical Modelling with Computer Extended Descriptive Geometry (CeDG): description and comparison with CAD, *Computer-Aided Design and Applications*, 18 (2), 272-284. <https://doi.org/10.14733/cadaps.2021.272-284>

Prado-Velasco, M., Ortiz Marín, R. (2021). Comparison of Computer Extended Descriptive Geometry (CeDG) with CAD in the Modeling of Sheet Metal Patterns, *Symmetry*, 13(4), 272-284. <https://doi.org/10.3390/sym13040685>.

Preiner, J. (2008). Introducing Dynamic Mathematics Software to Mathematics Teachers: the case of Geogebra, *Thesis*. (2008). DOI: 10.13140/RG.2.2.15003.05921

Stachel, H. (2007). The Status of Todays Descriptive Geometry Related Education (Cad/Cg/Dg) in Europe, *Journal of Graphic Science of Japan*, 41(1), 15-20.

DOI:10.5989/jsgs.41.Supplement1_15

Weisberg, D.E. (2008). Computer-Aided Design Strong Roots at MIT, in: *The Engineering Design Revolution: The People, Companies and Computer Systems That Changed Forever the Practice of Engineering*. Englewood (EE.UU), 28-52.

<http://cadhistory.net/>